Unified Developer Workspace with Context-Switch Analytics

Abstract:

In modern software development, developers face constant interruptions from switching between IDEs, communication platforms, issue trackers, documentation sites, and email clients. These context switches significantly impact productivity, code quality, and developer well-being. This project presents a **Unified Developer Workspace with Context-Switch Analytics** that consolidates essential development tools into a single interface while monitoring and analyzing the impact of interruptions on productivity.

The proposed system integrates multiple developer tools (Slack/Teams, GitHub/GitLab, Jira, email, and documentation) into a centralized workspace with a powerful universal search feature that queries across all connected applications simultaneously. For example, searching "bug #234" retrieves results from GitHub issues, Jira tickets, Slack conversations, and code comments in one place, eliminating constant application switching. An intelligent tracking module monitors context-switching patterns, captures interruption events, and correlates them with productivity metrics such as commit frequency, task completion time, and focused work duration.

The system supports role-based access for team leads, developers, and managers with appropriate permissions and privacy controls. Through this dual approach—actively reducing context switches via workspace unification while tracking remaining interruptions—the system helps development teams achieve focused work states and improve overall productivity with measurable, data-driven evidence. This project addresses a critical productivity challenge in software development through both preventive and analytical solutions.

Key Features:

1. **Unified Tool Integration** -- Consolidates Slack/Teams, GitHub/GitLab, Jira, email, and documentation into a single interface with seamless navigation.

2. **Universal Cross-App Search** -- Intelligent search bar that queries across all connected applications simultaneously, returning unified results with source attribution.

3. **Context-Switch Tracking Engine** -- Monitors application switches, interruption events, and focused work sessions with timestamp logging and pattern analysis.

4. **Role-Based Access Control** -- Different roles like Manager, Team Lead, and Developer with specific permissions and privacy safeguards.

5. **Analytics Dashboard** -- Displays comprehensive metrics including context-switch frequency reduction, peak interruption times, focused work duration, productivity improvements, and before/after comparison reports.

PROJECT TITLE

Web-Based Python Code Compiler

ABSTRACT

This project is a web-based application that allows users to write, compile, and run Python programs directly in a web browser. Users do not need to install Python or any software on their system. They can simply open the website, write Python code, and execute it online. The application takes Python code as input, sends it to the server for execution, and displays the output or error messages on the same web page. This system is especially useful for students, beginners, and learners who want to practice Python programming anytime and anywhere using a browser.

Key Features:

- **Python code editor in the browser**
- **Supports only Python language**
- **Execute Python programs online**

S. No	Roll Number	Name of the Student	Mobile Number
1	24B81A67C1	G.SRIKARUN	9491681251
2.	24B81A6766	K.Akshay kumar	6281937360
3.	24B81A6765	U.K.Abhiram	88978 30216

Coordinator Name(s):

1. Dr.Annapurna Gummadi
2. Mrs M.Nitya

Signature(s) of the coordinator(s):

MEME GENERATOR APP USING FLASK

ABSTRACT

Memes have become a popular and powerful way of expressing ideas, emotions, and humor on social media platforms. Creating memes manually using image editing tools can be time-consuming and requires technical skills. The Meme Generator App using Flask is designed to simplify the process of meme creation by allowing users to easily generate memes through a web-based interface.

This application enables users to upload images or select predefined templates and add custom text to generate memes instantly. Built using Python Flask framework, the system processes user input efficiently and displays the generated meme in real time. The application is simple, user-friendly, and demonstrates the practical use of Python and web development concepts for real-world applications.

Key Features:

User-Friendly Meme Creation – Allows users to generate memes easily without technical knowledge.

Image Upload and Template Support – Supports custom image uploads and predefined meme templates.

Real-Time Meme Generation – Instantly displays the generated meme after text input.

SPLITTING EXPENSES MANAGEMENT SYSTEM

ABSTRACT

Splitting expenses in a group is often difficult during trips, shared rooms, or group events. When expenses are calculated manually, it may cause confusion, mistakes, and arguments because everyone may not pay the same amount. To solve this problem, the Splitting Expenses Management System helps in managing shared expenses in a simple and automatic way.

In this system, users can create a group, add members, and enter expenses. The system automatically splits the amount equally or as required and clearly shows who has to pay whom. This saves time, avoids misunderstandings, and ensures fair and accurate expense sharing using python.

Key Features:

Group-Based Expense Tracking – Allows creation of groups and management of shared expenses among members.

Automatic Expense Calculation – Splits expenses equally or based on custom contributions with high accuracy.

Real-Time Balance Summary – Clearly displays pending balances and settlements between users.

Expense History Management – Maintains a detailed record of all transactions for future reference.

S. No	Roll Number	Name of the Student	Mobile Number
1	24B81A67C3	U. Uday Sai	9550796959
2	24B81A67B4	A. Shanmukh	7842128844
3	24B81A67A3	E. Nissy	9390483942
4	24B81A67A4	S. Poojitha	7989114719

SPAM EMAIL DETECTION

ABSTRACT

Spam emails are one of the major problems in digital communication, causing inconvenience, security threats, and data loss to users. This project focuses on designing and implementing a Spam Mail Detection System that works across different operating systems such as Android and iOS.

The proposed system automatically classifies incoming emails into spam and non-spam categories by analyzing email content, sender details, subject lines, and message patterns. The system uses machine learning techniques or rule-based filtering to detect spam emails effectively.

Abstraction for Spam Mail Detection in Different Operating Systems

1. Spam Mail Detection in Android Operating System

In the Android operating system, spam mail detection is implemented at both the application level and the system level. Email applications such as Gmail use built-in machine learning models to analyze incoming emails in real time. The system abstracts spam detection by examining features like sender reputation, email content, links, attachments, and user behavior (marking emails as spam or not).

Android provides APIs that allow apps to integrate cloud-based and on-device intelligence for spam filtering. The abstraction hides the complexity of classification algorithms, enabling the OS and apps to automatically filter spam emails into a separate folder, notify users about suspicious messages, and continuously improve detection accuracy through updates.

2. Spam Mail Detection in iPhone (iOS) Operating System

In the iOS operating system, spam mail detection focuses on privacy and security while maintaining efficiency. Apple Mail uses on-device processing and secure server-side analysis to classify emails. The abstraction treats each email as a set of encrypted features such as subject, sender patterns, message body, and attachments.

iOS restricts direct access to system resources, so spam detection is tightly controlled by the OS. User actions like reporting spam are abstracted into feedback signals that improve